

# Fiche de Révision : Fichiers Textes en C

## Gestion, Lecture, Écriture et Algorithmes

### 1. Concepts de Base

Pour manipuler des fichiers en C, on utilise la bibliothèque standard `<stdio.h>`.

- `FILE *` : Type pointeur représentant le *flux* (stream) du fichier.
- `EOF` : *End Of File* (fin de fichier). Constante entière (souvent -1) renvoyée par les fonctions de lecture quand la fin est atteinte ou qu'une erreur survient.

### 2. Ouverture et Fermeture

#### 2.1 Ouvrir un fichier : `fopen`

```
1 FILE *fichier = fopen("nom.txt", "mode");
```

Modes d'ouverture pour le texte :

- `"r"` (*read*) : Lecture seule. Échoue si le fichier n'existe pas.
- `"w"` (*write*) : Écriture seule. Crée le fichier ou l'efface s'il existe.
- `"a"` (*append*) : Ajout en fin de fichier. Crée le fichier s'il n'existe pas.
- `"r+"` : Lecture et écriture. (Le fichier doit exister).
- `"w+"` : Lecture et écriture. (Efface ou crée).
- `"a+"` : Lecture et ajout à la fin.

Toujours vérifier l'ouverture :

```
1 if (fichier == NULL) {  
2     perror("Erreur d'ouverture");  
3     return 1;  
4 }
```

#### 2.2 Fermer un fichier : `fclose`

Obligatoire pour libérer les ressources et vider les tampons (buffers).

```
1 fclose(fichier);
```

### 3. Lecture de Fichiers

#### 3.1 Caractère par caractère : `fgetc`

Lit un caractère et avance le curseur.

```
1 int c = fgetc(fichier);  
2 // type int pour pouvoir gérer EOF
```

#### 3.2 Ligne par ligne : `fgets`

Recommandé pour lire des chaînes de manière sécurisée.

```
1 char buffer[256];  
2 // Lit au max 255 char, s'arrête au \n  
3 fgets(buffer, 256, fichier);
```

#### 3.3 Lecture formatée : `fscanf`

Semblable à `scanf`, mais pour un fichier. Dangereux si le fichier est mal formaté.

```
1 int age; char nom[50];  
2 fscanf(fichier, "%s %d", nom, &age);
```

### 4. Écriture dans les Fichiers

#### 4.1 Caractère par caractère : `fputc`

```
1 fputc('A', fichier);
```

#### 4.2 Chaîne de caractères : `fputs`

N'ajoute pas de saut de ligne automatiquement.

```
1 fputs("Bonjour le monde\n", fichier);
```

#### 4.3 Écriture formatée : `fprintf`

Idéal pour écrire des données de divers types.

```
1 int annee = 2026;  
2 fprintf(fichier, "Nous sommes en %d.\n", annee);
```

### 5. Positionnement (Le Curseur)

Chaque flux a un indicateur de position (curseur).

- `long ftell(FILE *f)` : Renvoie la position actuelle en octets.
- `void rewind(FILE *f)` : Remet le curseur au début du fichier.
- `int fseek(FILE *f, long déplacement, int origine)` : Déplace le curseur.
- Origines : `SEEK_SET` (début), `SEEK_CUR` (actuelle), `SEEK_END` (fin).

### 6. Algorithmes Classiques et Exemples Complets

#### 6.1 1. Lire et afficher tout le contenu d'un fichier texte (Caractère par caractère)

Cette méthode est infaillible pour afficher exactement le contenu d'un fichier.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main() {
5     FILE *f = fopen("data.txt", "r");
6     if (f == NULL) {
7         printf("Erreur lors de l'ouverture.\n");
8         return 1;
9     }
10
11     int c;
12     // On lit jusqu'a rencontrer la fin du fichier (EOF)
13     while ((c = fgetc(f)) != EOF) {
14         putchar(c); // ou printf("%c", c);
15     }
16
17     fclose(f);
18     return 0;
19 }

```

## 6.2 2. Compter le nombre de lignes dans un fichier

On parcourt le fichier en comptant les sauts de ligne (`\n`).

```

1 #include <stdio.h>
2
3 int compter_lignes(const char *nom_fichier) {
4     FILE *f = fopen(nom_fichier, "r");
5     if (!f) return -1; // Code d'erreur
6
7     int c;
8     int lignes = 0;
9     while ((c = fgetc(f)) != EOF) {
10         if (c == '\n') {
11             lignes++;
12         }
13     }
14     fclose(f);
15     return lignes;
16 }

```

## 6.3 3. Copier le contenu d'un fichier vers un autre (Ligne par ligne)

Plus rapide que caractère par caractère pour du texte volumineux.

```

1 #include <stdio.h>
2
3 int main() {
4     FILE *source = fopen("source.txt", "r");
5     FILE *dest = fopen("copie.txt", "w");
6
7     if (source == NULL || dest == NULL) {
8         printf("Erreur d'ouverture des fichiers.\n");
9         if (source) fclose(source);
10        if (dest) fclose(dest);
11        return 1;
12    }
13
14    char ligne[1024]; // Tampon pour stocker la ligne lue
15
16    // fgets renvoie NULL quand EOF est atteint
17    while (fgets(ligne, sizeof(ligne), source) != NULL) {
18        fputs(ligne, dest);
19    }
20
21    printf("Copie terminee.\n");
22
23    fclose(source);
24    fclose(dest);
25    return 0;
26 }

```

## 7. Recherche dans un Fichier Texte

Pour effectuer des recherches efficaces, on combine généralement la lecture ligne par ligne (`fgets`) ou formatée (`fscanf`) avec les fonctions de manipulation de chaînes de la bibliothèque `<string.h>`.

## 7.1 Fonctions clés (<string.h>)

- `char *strstr(const char *chaine, const char *sous_chaine)` : Cherche la première occurrence de la `sous_chaine` dans la `chaine`. Renvoie un pointeur si trouvé, sinon `NULL`.
- `int strcmp(const char *s1, const char *s2)` : Compare deux chaînes de caractères. Renvoie 0 si elles sont exactement identiques.
- `char *strtok(char *str, const char *delim)` : Découpe une chaîne en "jetons" (tokens) selon un délimiteur. Très utile pour analyser des fichiers type CSV.

## 7.2 4. Rechercher un mot spécifique et afficher la ligne

Cet algorithme parcourt le fichier ligne par ligne et affiche chaque ligne contenant le mot recherché, avec son numéro de ligne. C'est le principe de la commande `grep` sous Linux.

```
1 #include <stdio.h>
2 #include <string.h>
3
4 void rechercher_mot(const char *nom_fichier, const char *mot_cle) {
5     FILE *f = fopen(nom_fichier, "r");
6     if (f == NULL) {
7         printf("Erreur d'ouverture du fichier.\n");
8         return;
9     }
10
11     char ligne[1024];
12     int numero_ligne = 1;
13     int occurrences = 0;
14
15     // Lecture sécurisée ligne par ligne
16     while (fgets(ligne, sizeof(ligne), f) != NULL) {
17         // strstr vérifie si le mot_cle est présent dans la ligne lue
18         if (strstr(ligne, mot_cle) != NULL) {
19             printf("Ligne %d : %s", numero_ligne, ligne);
20             occurrences++;
21         }
22         numero_ligne++;
23     }
24
25     if (occurrences == 0) {
26         printf("Le mot '%s' n'a pas été trouvé.\n", mot_cle);
27     } else {
28         printf("Total : %d occurrence(s) trouvée(s).\n", occurrences);
29     }
30
31     fclose(f);
32 }
```

## 7.3 5. Rechercher une donnée structurée (Lecture formatée)

Imaginons un fichier `etudiants.txt` organisé ainsi : ID Nom Note (ex : 102 Dupont 15.5). L'objectif est de retrouver la note et le nom d'un étudiant à partir de son ID.

```
1 #include <stdio.h>
2
3 void rechercher_etudiant_par_id(const char *nom_fichier, int id_recherche) {
4     FILE *f = fopen(nom_fichier, "r");
5     if (f == NULL) {
6         printf("Fichier introuvable.\n");
7         return;
8     }
9
10     int id_lu;
11     char nom[50];
12     float note;
13     int trouve = 0;
14
15     // fscanf lit les variables selon le format et renvoie le nombre d'éléments lus
16     // Tant qu'on arrive à lire 3 éléments correctement (ID, Nom, Note)
17     while (fscanf(f, "%d %49s %f", &id_lu, nom, &note) == 3) {
18         if (id_lu == id_recherche) {
19             printf("Étudiant trouvé ! Nom: %s | Note: %.2f/20\n", nom, note);
20             trouve = 1;
21             break; // On arrête la boucle puisqu'on a trouvé notre étudiant
22         }
23     }
24
25     if (!trouve) {
```

```
26     printf("Aucun étudiant avec l'ID %d n'a été trouvé.\n", id_recherche);
27 }
28
29 fclose(f);
30 }
```

#### 7.4 Astuce : Que faire si on veut modifier ce qu'on a cherché ?

Dans un fichier texte pur, on ne peut pas insérer ou supprimer du texte au milieu sans écraser la suite. La méthode classique consiste à :

1. Ouvrir le fichier original en lecture ("r").
2. Ouvrir un fichier temporaire en écriture ("w").
3. Lire ligne par ligne. Si la ligne est celle à modifier/supprimer, écrire la nouvelle ligne (ou ne rien écrire) dans le fichier temporaire. Sinon, copier la ligne telle quelle.
4. Fermer les deux fichiers.
5. Supprimer le fichier original (`remove()`) et renommer le fichier temporaire avec le nom de l'original (`rename()`).